

Lab 2 Report: Performance Analysis

gem5 Garnet Network-on-Chip Simulation

Name: _____
Student ID: _____

gem5 v23.0.0.1 July 2026

Abstract

This laboratory evaluates latency and throughput in the gem5 Garnet network model. Five synthetic traffic patterns are first swept over injection rates from 0.01 to 0.50 on a 64-node, 8×8 mesh. Controlled experiments then vary virtual channels, router pipeline latency, and link width. The results show that path length determines low-load latency, traffic placement determines the congestion point, sufficient virtual channels prevent premature blocking, and link width matters only when packets span multiple flits. Finally, the measured latency components, default input-buffer depth, and credit-based virtual-channel wormhole flow control are analyzed.

Contents

1	Experimental Method	2
2	Task 1: Synthetic Traffic Patterns	2
2.1	Traffic definitions	2
2.2	Latency-throughput results	2
3	Task 2: Controlled Network Parameters	4
3.1	Virtual channels per vnet	4
3.2	Router pipeline latency	5
3.3	Link width and packet serialization	6
4	Task 3: Latency, Buffers, and Flow Control	7
4.1	Latency components and bottlenecks	7
4.2	Default input-buffer depth	7
4.3	Default flow control	7
5	Reproduction and Validation	8
6	Conclusion	8

1 Experimental Method

The experiments use gem5 v23.0.0.1, built for the NULL ISA with the `Garnet_standalone` protocol. The global tick frequency and Ruby clock are both 2 GHz, while the synthetic traffic generators use the default 1 GHz system clock. Consequently, one traffic-generator cycle contains two Ruby cycles. Every point runs for 10000 Ruby cycles on 64 routers and 64 directory destinations arranged as an 8×8 Mesh_XY network.

The common command is represented by:

```
./build/NULL/gem5.opt \
  configs/example/garnet_synth_traffic.py \
  --network=garnet --num-cpus=64 --num-dirs=64 \
  --topology=Mesh_XY --mesh-rows=8 \
  --sim-cycles=10000 --injectionrate=<rate> \
  --synthetic=<pattern> --inj-vnet=<vnet>
```

The command-line injection rate is measured in packets/node/system-cycle. The offered load expressed per Ruby cycle is therefore

$$\lambda_{\text{offered}} = \frac{\text{injection rate}}{2}. \quad (1)$$

Achieved packet throughput is normalized consistently as

$$T = \frac{N_{\text{packets received}}}{64 \text{ nodes} \times 10000 \text{ Ruby cycles}}. \quad (2)$$

Thus the largest requested injection rate, 0.50, offers at most 0.25 packets/node/Ruby-cycle. All Task 1 experiments and the VC/router experiments in Task 2 use vnet 0, which carries 8-byte, single-flit control packets.

2 Task 1: Synthetic Traffic Patterns

2.1 Traffic definitions

For a node at coordinate (x, y) in an 8×8 mesh, the five traffic patterns select destinations as follows. Node identifiers use $\text{id} = 8y + x$.

Table 1: Destination mappings and expected Manhattan hop counts.

Pattern	Destination rule	Interpretation	$E[H]$
Uniform random	random node in $[0, 63]$	spatially distributed	5.25
Neighbor	$((x + 1) \bmod 8, y)$	next node in the row	1.75
Tornado	$((x + 3) \bmod 8, y)$	fixed near-half-row offset	3.75
Transpose	(y, x)	reflection across the diagonal	5.25
Shuffle	6-bit cyclic left rotation	deterministic permutation	4.00

The modulo operation chooses the destination but does not create a physical wraparound link. For example, a neighbor packet from $x = 7$ to $x = 0$ traverses seven mesh links. Uniform random chooses a new destination for every packet; the other patterns map each source to a fixed destination.

2.2 Latency-throughput results

All five patterns were simulated at 50 rates from 0.01 through 0.50, producing 250 data points. Figure 1 plots average packet latency against achieved packet throughput.

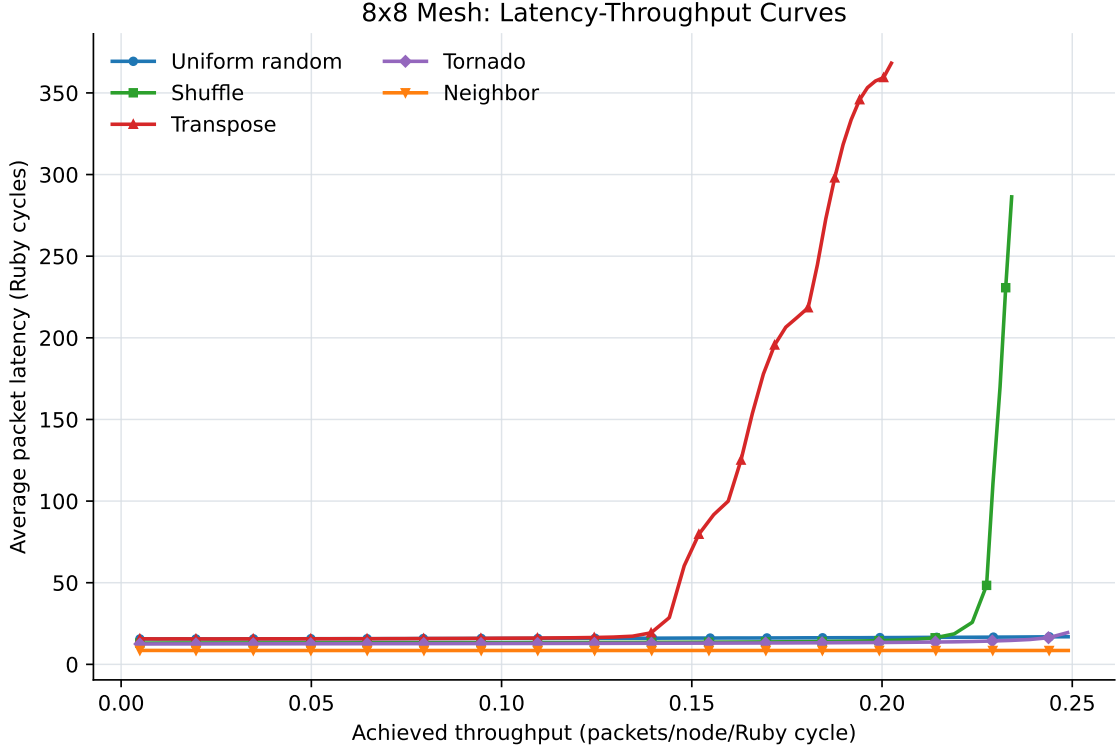


Figure 1: Task 1 latency-throughput curves for five traffic patterns.

At injection rate 0.01, queueing latency is 2 cycles for every pattern. The measured total latency is approximately

$$L_{\text{low}} \simeq 5 + 2E[H], \quad (3)$$

so the low-load ordering follows average path length. The measurements closely match the exact theoretical hop counts.

Table 2: Low-load measurements at injection rate 0.01.

Pattern	Packet latency	Measured hops	Theoretical hops
Neighbor	8.531	1.766	1.750
Tornado	12.469	3.726	3.750
Shuffle	13.025	4.006	4.000
Uniform random	15.558	5.269	5.250
Transpose	15.662	5.324	5.250

The high-load results reveal a separate effect: equal average path length does not imply equal throughput. Uniform random distributes traffic statistically, whereas transpose concentrates deterministic XY paths on particular links. Transpose reaches twice its low-load latency at rate 0.30; shuffle reaches this threshold at rate 0.46. Uniform random, tornado, and neighbor do not reach the same threshold within the requested range.

Table 3: Measurements at the maximum injection rate of 0.50.

Pattern	Throughput	Queue latency	Total latency	Average hops
Uniform random	0.248914	2.000	16.905	5.248
Shuffle	0.234073	258.784	286.325	3.984
Transpose	0.202458	343.439	368.197	4.026
Tornado	0.248770	4.929	19.425	3.747
Neighbor	0.248956	2.000	8.495	1.748

The apparent decline in transpose hop count under overload is not a routing change. XY routing and the $(x, y) \rightarrow (y, x)$ mapping remain fixed. Garnet computes average hops only from flits that arrive. Under severe backpressure, long, heavily contended flows are less likely to enter and complete within the finite measurement window, so completed short paths are overrepresented.

3 Task 2: Controlled Network Parameters

Task 2 changes one parameter at a time while retaining the default baseline of four VCs per vnet, one-cycle routers, and 128-bit links. Uniform random is used as a distributed workload and transpose as a hotspot-prone workload.

Table 4: Task 2 sweep matrix.

Experiment	Values	Packet class
VCs per vnet	1, 2, 4, 8	vnet-0 control
Router latency	1, 2, 4 cycles	vnet-0 control
Link width control	64, 128, 256 bits	vnet-0, 8 bytes
Link width data	64, 128, 256 bits	vnet-2, 72 bytes

With 50 rates, two patterns, and 13 parameter variants, the complete Task 2 dataset contains 1300 points.

3.1 Virtual channels per vnet

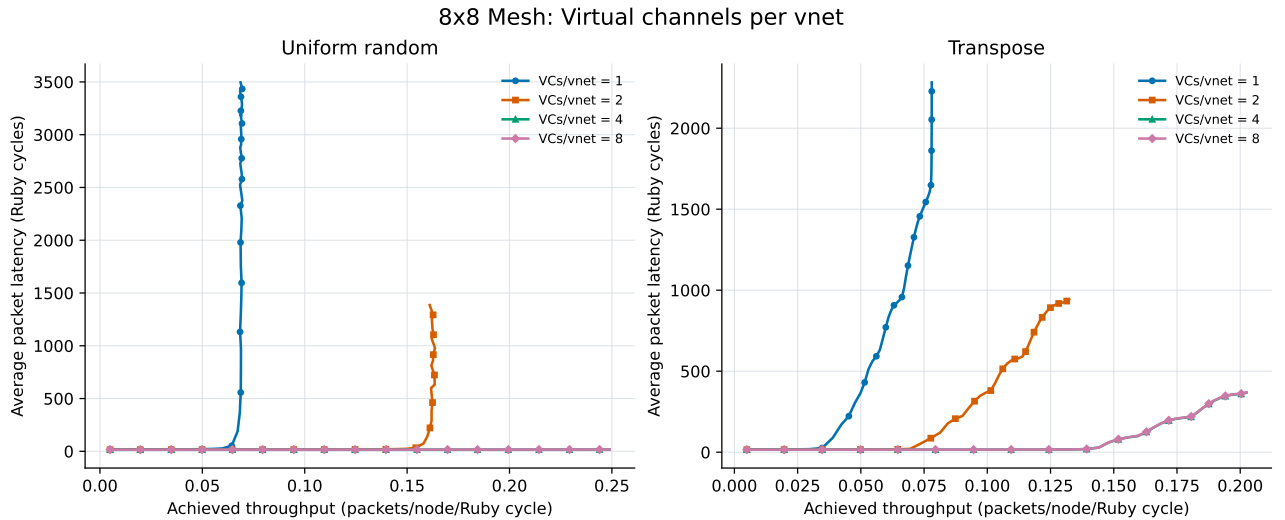


Figure 2: Effect of virtual channels per vnet.

Virtual channels do not increase the one-flit-per-cycle physical-link limit. They reduce head-of-line blocking, add independent buffer slots, and allow other ready flows to use a link while one flow waits

for downstream credit. This improves utilization and delays congestion.

Table 5: VC sweep: maximum observed throughput and $2\times$ latency rate.

Pattern	Maximum throughput				$2\times$ latency rate			
	VC1	VC2	VC4	VC8	VC1	VC2	VC4	VC8
Uniform random	.070	.164	.249	.249	.13	.31	–	–
Transpose	.078	.132	.202	.202	.08	.15	.30	.30

Four VCs are sufficient for these control-packet workloads. Increasing from four to eight changes observed throughput by less than 5×10^{-6} . At that point link capacity and traffic placement, rather than VC availability, are the dominant limitations.

3.2 Router pipeline latency

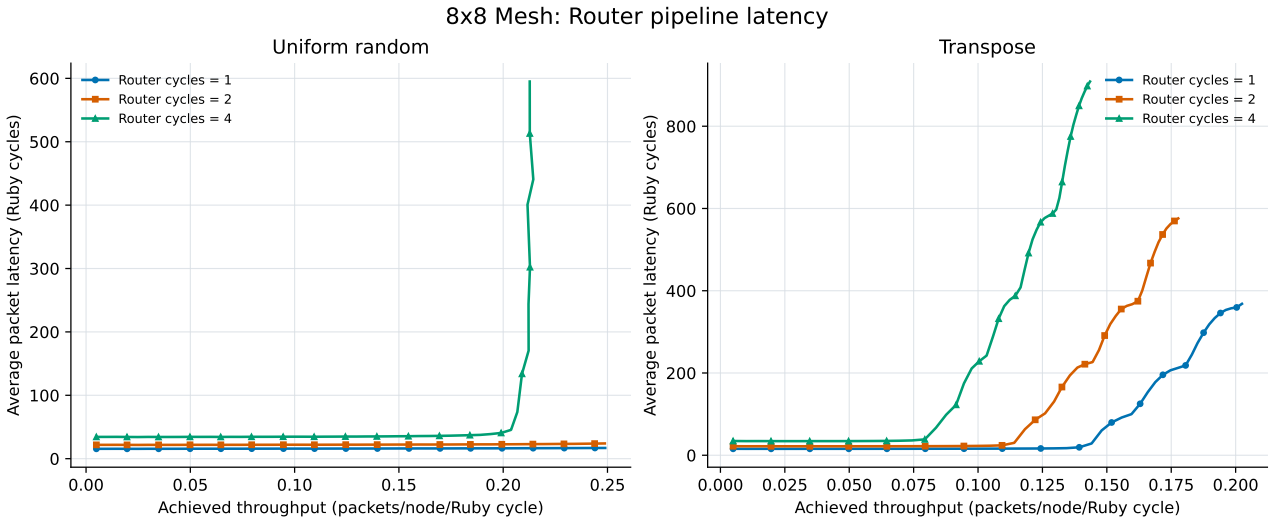


Figure 3: Effect of router pipeline latency.

Increasing router latency raises the zero-load cost at every traversed router. For uniform random, low-load latency increases from 15.56 to 21.82 and 34.35 cycles as router latency changes from one to two and four cycles. Transpose shows the same progression: 15.66, 21.98, and 34.62 cycles. The increment is close to

$$\Delta L = (L_{\text{router}} - 1)(E[H] + 1), \quad (4)$$

because each packet pays the additional pipeline stages at every router along its path. Longer pipelines also delay credit return. For transpose, the $2\times$ -latency knee moves from injection rate 0.30 to 0.24 and 0.18, while maximum observed throughput falls from 0.202 to 0.178 and 0.143.

3.3 Link width and packet serialization

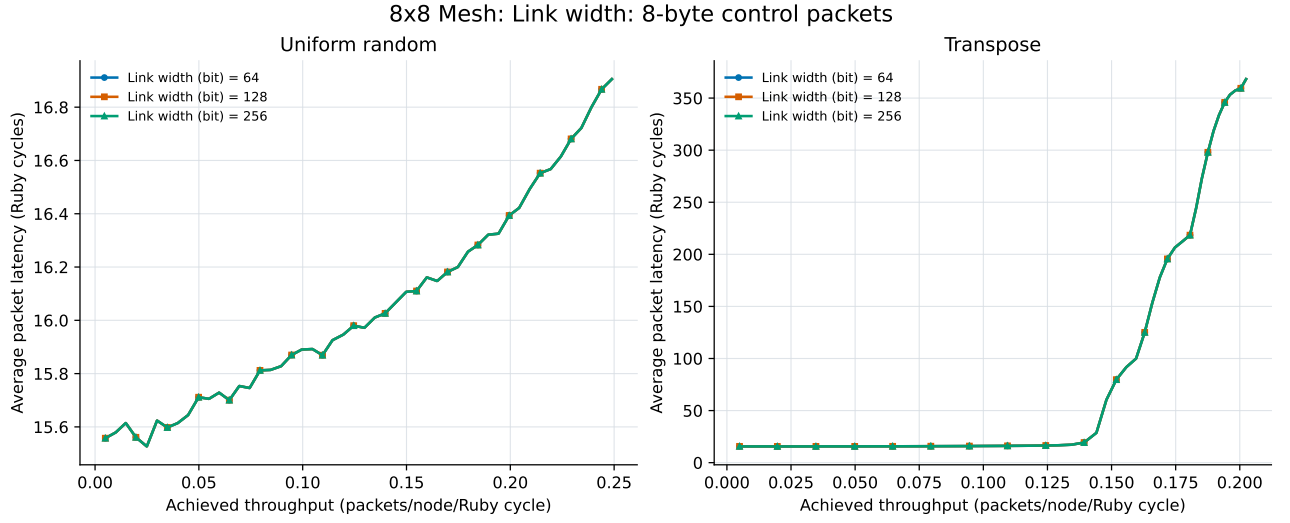


Figure 4: Link-width sweep with 8-byte control packets.

The 64-, 128-, and 256-bit control-packet curves are exactly equal. An 8-byte packet occupies one flit at all tested widths, and Garnet still transfers one flit per link per cycle. Unused bits cannot carry a second packet during the same cycle.

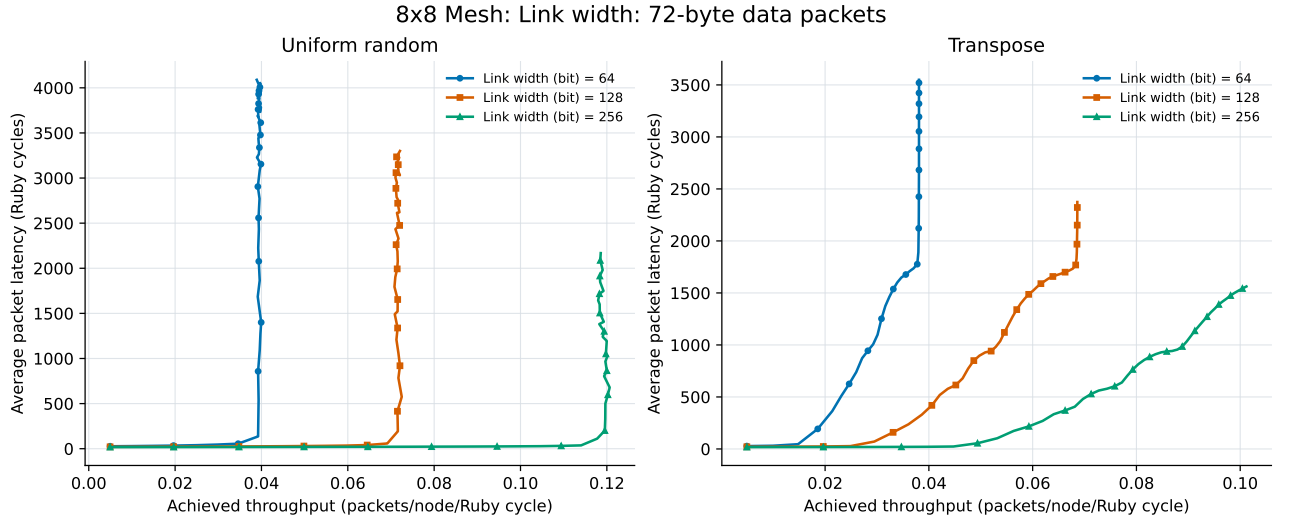


Figure 5: Link-width sweep with 72-byte data packets.

For 72-byte packets, width changes the number of flits and hence serialization time and offered flit load.

Table 6: Data-packet link-width results.

Width	Flits/packet	Low-load latency	Uniform peak	Transpose peak
64 bit	9	27.030	0.03994	0.03810
128 bit	5	20.906	0.07249	0.06861
256 bit	3	17.742	0.12065	0.10120

For uniform random, multiplying peak packet throughput by flits per packet gives 0.359, 0.362, and 0.362 flits/node/cycle. The nearly constant flit rate confirms that wider links improve packet

throughput by dividing the same network flit capacity among fewer flits per packet. Transpose remains lower because its deterministic paths concentrate load on fewer links.

4 Task 3: Latency, Buffers, and Flow Control

4.1 Latency components and bottlenecks

Garnet reports packet latency as

$$L_{\text{packet}} = L_{\text{queueing}} + L_{\text{network}}. \quad (5)$$

Queueing latency is the sum of source-side waiting before network injection and destination-side waiting after network dequeue. Network latency covers the interval inside Garnet, including packet serialization, route and VC allocation, switch allocation, router pipelines, link traversal, router-buffer contention, and all hops.

At low load, queueing is approximately two cycles, so path length, router latency, link latency, and serialization dominate. At high load, occupied VCs and buffers prevent credit return; backpressure reaches the source and endpoint queueing grows rapidly. Table 7 illustrates the change at rate 0.50.

Table 7: Latency decomposition for representative Task 1 patterns.

Pattern	Queueing latency	Network latency	Total latency
Uniform random	2.000	14.905	16.905
Shuffle	258.784	27.541	286.325
Transpose	343.439	24.758	368.197

Thus network traversal dominates unsaturated uniform-random traffic, whereas endpoint queueing caused by network backpressure dominates saturated shuffle and transpose. Narrow links add serialization at low load and multiply flit contention at high load. Wider links eliminate this penalty for multi-flit data packets but cannot improve an already single-flit control packet.

4.2 Default input-buffer depth

The relevant defaults in `src/mem/ruby/network/garnet/GarnetNetwork.py` are:

```
vcs_per_vnet = 4
buffers_per_ctrl_vc = 1
buffers_per_data_vc = 4
```

Therefore each router input port provides four flit slots for each control vnet and sixteen slots for the data vnet:

$$B_{\text{control}} = 4 \text{ VCs} \times 1 = 4 \text{ flits}, \quad B_{\text{data}} = 4 \text{ VCs} \times 4 = 16 \text{ flits}. \quad (6)$$

Garnet standalone has two control vnets and one data vnet, giving 24 total flit slots per physical input port when all three are counted. Task 1 uses only vnet 0, so its relevant default capacity is four one-flit control VCs.

4.3 Default flow control

Garnet uses credit-based virtual-channel wormhole flow control. The upstream router maintains one credit for every free slot in the downstream VC. Sending a flit consumes a credit; when the downstream buffer releases that flit, a credit travels upstream. A head flit reserves a VC, body flits follow it, and the tail or head-tail flit returns a free signal that releases the VC.

Wormhole operation allows different flits of one packet to occupy several routers simultaneously, so a router need not buffer the complete packet. It also creates backpressure: if the head cannot

advance, body and tail flits remain distributed along the path and eventually block upstream injection. Multiple VCs reduce this blocking by allowing another ready packet to use the same physical port, but arbitration still selects at most one flit per output link per cycle.

5 Reproduction and Validation

The Task 1 and Task 2 runners append completed points and resume missing points. The commands used to reproduce the versioned CSV files and figures are:

```
./venv/bin/python lab/lab2/task1/run_sweep.py
./venv/bin/python lab/lab2/task1/plot_results.py

./venv/bin/python lab/lab2/task2/run_sweep.py
./venv/bin/python lab/lab2/task2/plot_results.py
```

Task 1 contains 250 unique points and Task 2 contains 1300 unique points. All values are finite, all runs report `simTicks=10000` and `simFreq=2000000000`, and no Task 2 log contains a fatal, panic, or deadlock message.

6 Conclusion

The measurements separate three causes of NoC performance. Hop count and pipeline depth set low-load latency; traffic placement and finite buffering set the congestion point; and packet flit count converts link-level capacity into packet throughput. Virtual channels improve utilization but cannot exceed physical bandwidth. Router latency accumulates per hop and lengthens the credit loop. Link width is valuable for multi-flit data traffic but irrelevant after a control packet already fits in one flit. Under saturation, backpressure and queueing, rather than the uncongested traversal time, dominate total latency.